LeanFM, OAuth, and the Legal Global Session Type

Conversation transcript

9 November 2026

This export contains the user-facing conversation and progress updates. System instructions, repository setup messages, and raw tool payloads are omitted. The OAuth diagram is regenerated from the same protocol in a light theme for print. An appendix records the checked correction completed while this export was requested.

1. LeanFM through Claude Desktop

User

i have been using leanFM from Claude Desktop. it has an mcp server. the idea is to grant software engineering powers to the console. It uses a branch of Session Types under CTL to describe a system of requirements.

Assistant

LeanFM gives Claude Desktop a formal requirements layer through MCP: it can describe a system using session types under CTL, then use those requirements to guide software engineering work.

The useful distinction is that session types describe the permitted interactions, while CTL expresses properties across possible execution paths—what must always hold, what can eventually happen, and what must never happen. Together, they let the console reason about behavior before translating it into an implementation specification and code.

2. MCP availability

User

can you use the leanFM MCP server?

Assistant — progress

I'll check whether LeanFM's MCP tools are available in this session.

Assistant

Yes—the LeanFM MCP server is connected here. I successfully called `requirements_interrogation`.

I can access its CTL checks, session-type validation, generated diagrams, simulation, trace conformance, code skeletons, and artifact validation. I can use those alongside this workspace's files to carry requirements through to implementation.

3. OAuth

User

can you show me a diagram of how OAuth works?

Assistant — progress

I'll model OAuth's authorization-code flow with PKCE in LeanFM, then show the generated interaction diagram.

I'm using the repository's LeanFM skill to generate the diagram from a checked protocol. The example will show how an app gets permission to call an API without receiving the user's password.

Generated visual

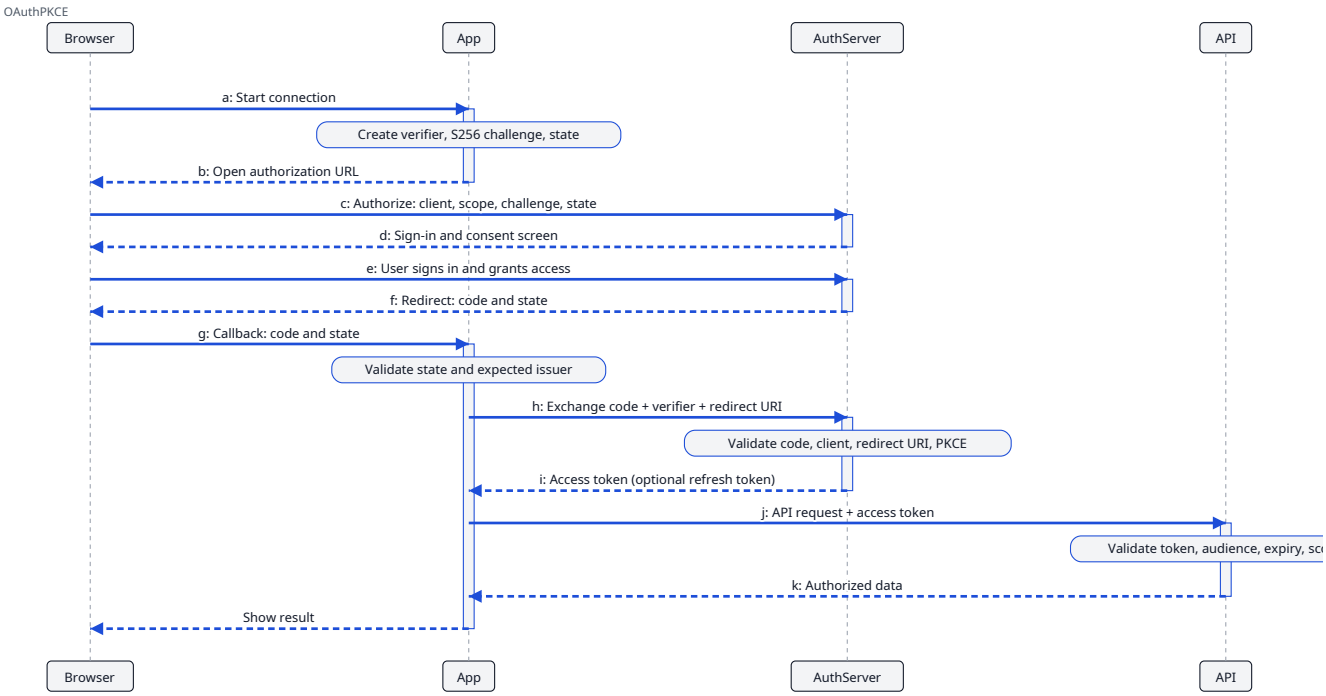


Figure 1: OAuth 2.0 authorization-code flow with PKCE. Generated by LeanFM from the checked protocol.

Browser = user's browser; App = client backend; AuthServer = authorization server; API = resource server. This is a successful grant; failure and refresh flows are omitted. Validation labels describe required checks, not cryptographic proofs.

Diagram provenance: prompt "can you show me a diagram of how OAuth works?"; question "How does a client obtain and use delegated API access?"; event source OAuthPKCE; reducer LeanFM session_diagram; unit ordered protocol messages, not elapsed time; boundary one successful authorization-code exchange and API request, with TLS assumed.

Assistant

OAuth grants an app limited access to an API without giving it your password.

The **authorization code** is exchanged for an **access token**. PKCE binds that exchange to the app's original secret verifier, helping prevent use of a stolen code. OAuth security standard

LeanFM found no structural issues in this example; that checks the modeled interactions, not the security of an implementation.

4. Deducing the legal session type

User

chapter 12 is a legal example… the Session Type is not written out. can you deduce it?

https://rfielding.github.io/leanFM-dark.pdf

Assistant — progress

I'll read chapter 12, infer the roles and interactions from the legal example, and check the resulting session type with LeanFM. I'll distinguish what the chapter states from any assumptions needed to complete the protocol.

Chapter 12 provides four histories, so it supports an inferred protocol, but not a unique one. The key is to keep unlawful behavior—including late return and rent offset—in the admissible branches.

I also found two details that matter: the lost-letter history ends without the landlord learning which branch occurred, and the hardship history's source list places day 330 after day 365. I'll preserve the distinction between a recorded history and a protocol each participant could execute.

Assistant

Yes—with one important qualification: **the four histories determine a candidate grammar, but not a unique, fully projectable session type.**

The inferred structure is:

```
Landlord → Tenant: LeaseSigned
Tenant → Landlord: DepositPaid
Landlord → Registry: DepositProtected

choice at Tenant:
  ordinary ending:
    Tenant → Landlord: LeaseEnded

    choice at Tenant:
      address delivered:
        Tenant → Landlord: ForwardingAddress

        choice at Landlord:
          itemised return:
            Landlord → Tenant: DeductionsItemized
            Landlord → Tenant: DepositReturned

          late return and reparation:
            Landlord → Tenant: DepositReturned
            Tenant → Court: ClaimFiled
            Court → Landlord: Judgment
            Landlord → Tenant: PenaltyPaid

      address lost:
        Tenant → Post: AddressPosted
        Tenant: BelievesViolated

  hardship:
    Tenant → Landlord: HardshipRequest
    Landlord → Tenant: RentOffsetAgreed
    Tenant → Landlord: LeaseEnded
    Tenant → Landlord: ForwardingAddress
    Landlord → Tenant: DepositReturned
```

I checked the actual LeanFM syntax. Each of the **four individual scenario types passes**. The combined type reports:

> Landlord cannot tell which branch Tenant chose.

That exposes a real modeling gap: after LeaseEnded, the landlord cannot distinguish a forthcoming address from a letter lost in the post. The recorded history stops; the landlord has received no event establishing that its session has ended. Adding a loss notification would change the example's knowledge model.

The statute and moral code belong **over this grammar**: late return and rent offset must remain possible so they can be classified as violations. Dates and knowledge-triggered obligations require

the chapter's additional evaluation layer. Chapter 12

I saved the candidate session type, four checked scenarios, and inference qualifications under `book/generated/legal-session-inference/`.

5. The global type as text

User

get me the text of the global session type

Assistant

```
protocol DepositHistories (Tenant, Landlord, Registry, Post, Court) {
  lease: LeaseSigned from Landlord to Tenant;
  paid: DepositPaid from Tenant to Landlord cause lease;
  DepositProtected from Landlord to Registry cause paid;
  choice {
    LeaseEnded from Tenant to Landlord;
    choice {
      address: ForwardingAddress from Tenant to Landlord;
      choice {
        DeductionsItemized from Landlord to Tenant cause address;
        DepositReturned from Landlord to Tenant cause address;
      } or {
        returned: DepositReturned from Landlord to Tenant cause address;
        claim: ClaimFiled from Tenant to Court cause returned;
        judgment: Judgment from Court to Landlord cause claim;
        PenaltyPaid from Landlord to Tenant cause judgment;
      }
    } or {
      AddressPosted from Tenant to Post;
      BelievesViolated at Tenant;
    }
  } or {
    hardship: HardshipRequest from Tenant to Landlord;
    RentOffsetAgreed from Landlord to Tenant cause hardship;
    LeaseEnded from Tenant to Landlord;
    address2: ForwardingAddress from Tenant to Landlord;
    DepositReturned from Landlord to Tenant cause address2;
  }
}
```

This is the inferred global type. LeanFM flags one projection issue: `Landlord cannot tell which branch Tenant chose` in the delivered-address versus lost-letter choice.

6. Fixing the projection issue

User

ok. can you fix that issue?

Assistant — progress

I'll check how to represent the lost-letter history as an unfinished interaction, so the landlord doesn't need a notification that would change what it knows.

The revised type passes LeanFM's checks. It adds one explicit recovery assumption: after the recorded lost-letter history, the tenant can resend the address directly. The landlord waits for `ForwardingAddress` in either case, so it needn't know whether a letter was lost.

The original lost-letter history remains a valid unfinished prefix. I'm checking that it cannot be mistaken for a completed session, and that a refund before address delivery is rejected.

7. Export request

User

turn this conversation into a pdf

Assistant — progress

I'll save the checked correction, then make a PDF of our conversation, including the OAuth diagram and both versions of the legal session type.

Appendix A. Checked correction

The following is the completed correction, added as an export appendix rather than attributed to an earlier assistant reply.

Scope and assumption. The chapter's lost-letter observation ends before address receipt. The revised protocol preserves that observation as an incomplete prefix and adds a possible later direct resend. Completion requires that recovery; the model does not prove the tenant will actually re-send. Permanent waiting remains an unfinished observation, not a completed branch. No delivery notification or forwarding address is invented within the original recorded horizon.

Positive invariant. The landlord follows the same receive-and-settle behavior whether an earlier postal attempt occurred or not.

Negative requirements. The lost-letter prefix must not count as complete. Deposit return must not precede receipt of the forwarding address in that branch. The correction must not remove the chapter's unlawful outcomes.

Control and evidence. Both address branches have the same landlord projection; the recovery branch requires ForwardingAddress before settlement. Structural checks pass, the three completed source scenarios still conform, and negative conformance checks reject premature completion and return.

```
protocol DepositHistories (Tenant, Landlord, Registry, Post, Court) {
  lease: LeaseSigned from Landlord to Tenant;
  paid: DepositPaid from Tenant to Landlord cause lease;
  DepositProtected from Landlord to Registry cause paid;
  choice {
    LeaseEnded from Tenant to Landlord;
    choice {
      address: ForwardingAddress from Tenant to Landlord;
      choice {
        DeductionsItemized from Landlord to Tenant cause address;
        DepositReturned from Landlord to Tenant cause address;
      } or {
        returned: DepositReturned from Landlord to Tenant cause address;
        claim: ClaimFiled from Tenant to Court cause returned;
        judgment: Judgment from Court to Landlord cause claim;
        PenaltyPaid from Landlord to Tenant cause judgment;
      }
    } or {
      AddressPosted from Tenant to Post;
      BelievesViolated at Tenant;
      -- Proposed recovery: a later direct resend, beyond the recorded horizon.
      address: ForwardingAddress from Tenant to Landlord;
      choice {
        DeductionsItemized from Landlord to Tenant cause address;
        DepositReturned from Landlord to Tenant cause address;
      } or {
        returned: DepositReturned from Landlord to Tenant cause address;
        claim: ClaimFiled from Tenant to Court cause returned;
        judgment: Judgment from Court to Landlord cause claim;
        PenaltyPaid from Landlord to Tenant cause judgment;
      }
    }
```

```
  } or {
    hardship: HardshipRequest from Tenant to Landlord;
    RentOffsetAgreed from Landlord to Tenant cause hardship;
    LeaseEnded from Tenant to Landlord;
    address2: ForwardingAddress from Tenant to Landlord;
    DepositReturned from Landlord to Tenant cause address2;
  }
}
```

Validation

- `session_check`: no issues.
- `session_review`: no findings; structural review, no HTTP routes, liveness and role checks.
- Itemised return: conforms, 7 events.
- Late return and penalty: conforms, 9 events.
- Hardship offset: conforms, 8 events, ordered consistently with the source timestamps.
- Lost-letter history with incomplete traces allowed: conforms, 6 events, session still running.
- Lost-letter history presented as complete: rejected; ForwardingAddress still expected.
- Recovery by direct resend followed by itemised return: conforms, 9 events.
- DepositReturned before ForwardingAddress: rejected.

These checks concern interaction structure and conformance. They do not establish compliance with the invented statute, moral code, deadlines, or amounts.

Source qualifications retained

Finite histories do not uniquely determine a protocol. The reconstruction serializes one ordering of events; it does not prove every sequential order is mandatory. In the source, hardship events at days 330 and 332 appear in the list after LeaseEnded at day 365; the reconstruction uses chronological order consistent with predecessor constraints. BelievesViolated is represented as a local Tenant event.

The source's `act2` evaluator checks DepositReturned rather than DeductionsItemized independently. Both occur on day 372 in the itemised example, so that example does not expose the difference from the prose rule.

Files

- `book/generated/legal-session-inference/DepositHistories.leanfm`: corrected global protocol.
- `book/generated/legal-session-inference/Scenarios.leanfm`: four individually checked original scenarios.
- `book/generated/legal-session-inference/validation.json`: correction conformance evidence.
- `LegalDepositExample.lean`: original chapter example source.

Sources: published book, chapter 12, local chapter example, and LeanFM MCP validation results.